

API REST COM JAVA SPRING

MINISTRADO POR

AUGUSTO MACEDO

DIEGO GONÇALVES MOTA



Ferramentas necessárias

- ◆ 1. IDE / Editor de Código
 - IntelliJ IDEA Community (recomendado)
 - ou VS Code com extensões (Java extension pack + Spring boot extension pack + Spring boot dashboard + Lombok)
- ◆ 2. Java Development Kit (JDK)
 - Versão 17 (LTS) ou compatível com Spring Boot (usar o comando no cmd: **java -version**)
- ◆ 3. Spring Initializr
 - Site: <https://start.spring.io>
 - Usado para gerar o projeto base
- ◆ 4. Ferramenta para Testar APIs
 - Postman ou Insomnia
- ◆ 5. Banco de Dados
 - PostgreSQL online (**Supabase**)



O que é uma API REST?

API: INTERFACE PARA COMUNICAÇÃO ENTRE SISTEMAS

REST (REPRESENTATIONAL STATE TRANSFER):

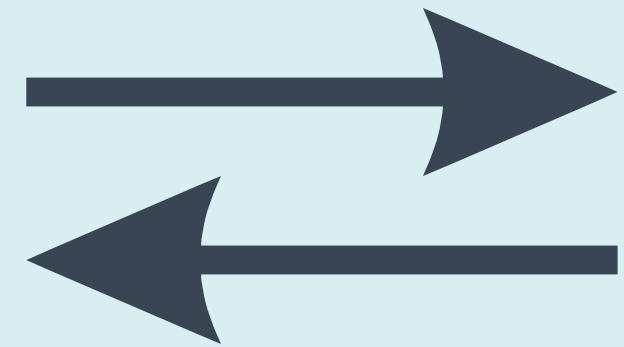
- BASEADO EM RECURSOS (/USERS, /PRODUCTS)
- UTILIZA HTTP COMO PROTOCOLO
- MÉTODOS: GET, POST, PUT, DELETE
- BENEFÍCIOS: SIMPLICIDADE, ESCALABILIDADE, PADRÃO AMPLAMENTE ADOTADO

Os 3 Pilares do REST

Um estilo arquitetural que define como as APIs devem ser construídas para serem escaláveis e padronizadas.

Pilar	Conceito	Impacto na API
1. Stateless (Sem Estado)	O servidor não guarda informações de sessão sobre o cliente entre as requisições.	Escalabilidade Máxima: Qualquer servidor pode atender a qualquer cliente a qualquer momento.
2. Cliente-Servidor	Separação de responsabilidades entre quem pede (cliente) e quem executa (servidor).	Independência de Evolução: O front-end e o back-end podem ser atualizados separadamente.
3. Interface Uniforme	Uso de um sistema de comunicação padronizado (URIs e Métodos HTTP).	Simplicidade e Desacoplamento: A API se torna intuitiva e fácil de ser usada por qualquer cliente (Web, Mobile, etc.).

Request e Response



Métodos

GET

POST

PUT

DELETE

PATCH

HTTP Status Codes mais usados em APIs REST

Ranges

1xx (Informativo): servidor recebeu a requisição, ainda processando (quase nunca usado em APIs).

2xx (Sucesso): operação realizada com sucesso.

3xx (Redirecionamento): cliente deve fazer nova requisição em outro lugar.

4xx (Erro do Cliente): problema na requisição enviada.

5xx (Erro do Servidor): falha do servidor ao processar.

HTTP Status Codes mais usados em APIs REST

Principais Códigos

◆ **Sucesso (2xx)**

200 OK → requisição bem sucedida (GET, PUT).

201 Created → recurso criado (POST).

204 No Content → sucesso, mas sem corpo (DELETE).

◆ **Erro do Cliente (4xx)**

400 Bad Request → dados inválidos.

401 Unauthorized → precisa autenticação.

403 Forbidden → autenticado, mas sem permissão.

404 Not Found → recurso não existe.

422 Unprocessable Entity → dados corretos no formato, mas inválidos na lógica.

◆ **Erro do Servidor (5xx)**

500 Internal Server Error → erro inesperado.

503 Service Unavailable → servidor indisponível/fora do ar.

O que é JSON?

Spring Framework:

- JSON (JavaScript Object Notation)
- Formato leve de troca de dados entre sistemas.
- Baseado em pares chave-valor.
- Muito usado em APIs REST.

```
{  
  "id": 1,  
  "nome": "Pizza",  
  "preco": 39.90,  
  "disponivel": true,  
  "ingredientes": ["queijo", "tomate", "orégano"]  
}
```


O que é Spring e Spring Boot?

Spring Framework:

- Framework Java completo
- Suporte a módulos (MVC, Data, Security)
- Flexível, mas exige muita configuração manual



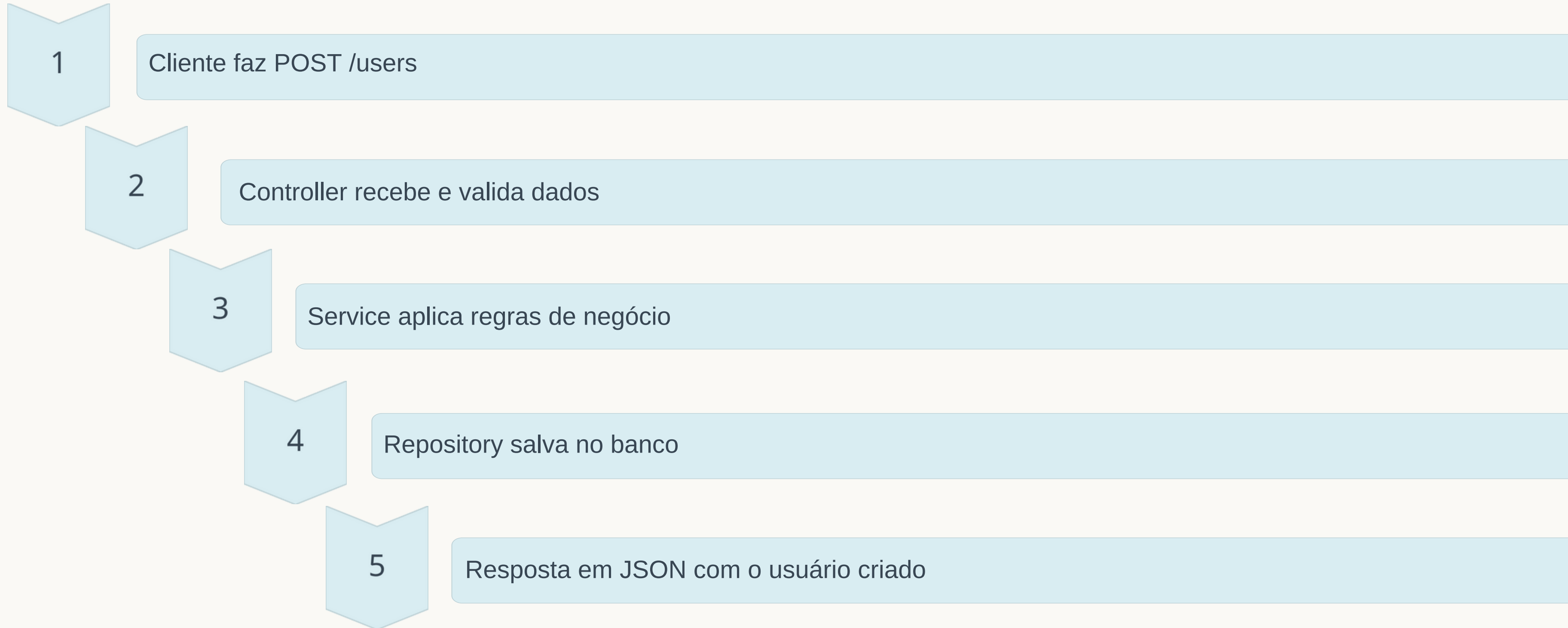
Spring Boot:

- Extensão do Spring para simplificar o desenvolvimento
- Configuração automática e servidor embutido
- Dependências facilitadas (starters)
- Ideal para criar APIs REST rápidas e escaláveis



Exemplo Prático

Fluxo de criação de usuário:



A decorative illustration on the left side of the slide. It features a light green folder with several documents inside. One document is white with a red heart and the letter 'M'. Another is orange with the text 'Spring Boot'. The folder is surrounded by various flowers and leaves in shades of blue, pink, and yellow.

Estrutura do Projeto

src/main/java/...

- controller/

- service/

- repository/

- domain/

Insomnia



O que é?

- Ferramenta gratuita para testar APIs REST.
- Permite enviar requisições HTTP (GET, POST, PUT, DELETE).
- Mostra a resposta em JSON, com status code e tempo de execução.

Por que iremos usar

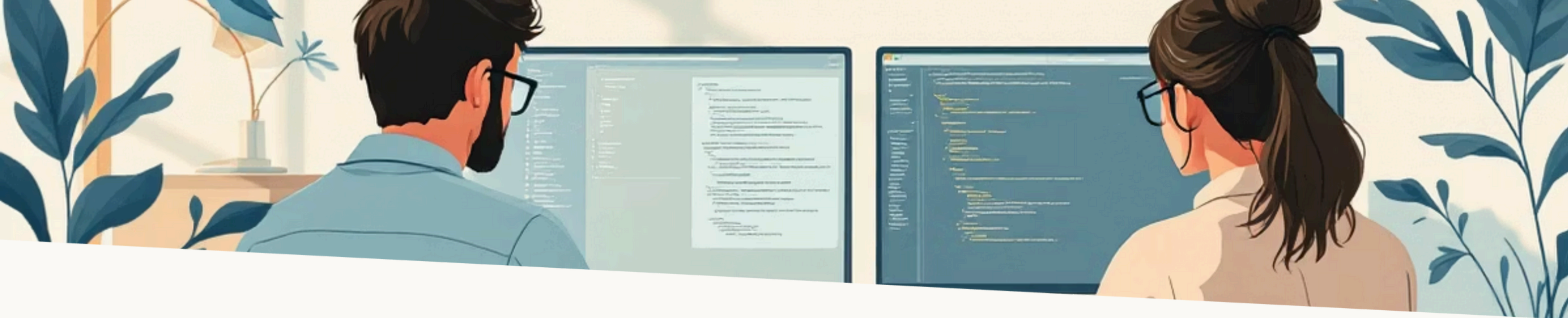
- Testar endpoints sem precisar criar front-end.
- Visualizar respostas de forma clara (JSON formatado).
- Simular erros e validar regras de negócio.

Funcionalidades básicas

- Criar requisições e coleções organizadas.
- Configurar headers (ex.: Content-Type, Authorization).
- Mostra a resposta em JSON, com status code e tempo de execução.

Alternativas

- Postman (mais famoso, mas mais pesado).
- cURL (via terminal, mas menos amigável).



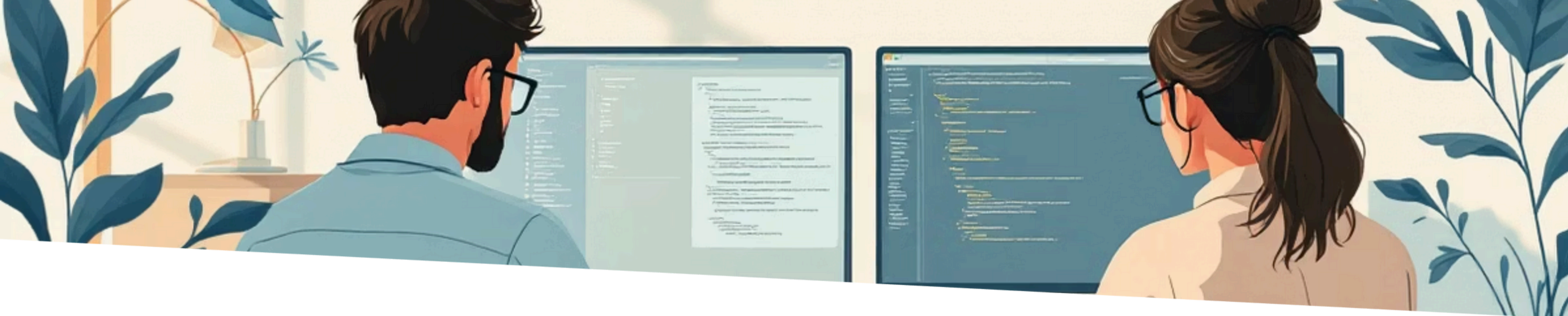
Conclusão e Próximos Passos

- Revisão dos conceitos:
 - API REST
 - Spring e Spring Boot
 - Arquitetura em camadas
- Próxima etapa: Codificação prática 🚀

Desafio

Criar um usuário

- Criar a entidade usuário com os campos id, email, senha e telefone
- Criar rotas de GET e POST de usuários
 - Criar um novo controller
 - Criar um repository
 - Criar DTOs
- Criar um novo usuário com seu nome e email pelo insomnia



Recapitulando...

- Criamos uma aplicação Spring Boot em camadas
- Controller → Repository → Domain
- Dois DTOs criados para realizar a manipulação dos dados
- Todas as operações CRUD foram feitas
- Endpoints de GET, POST, PATCH e DELETE
- Criar User e endpoints de GET e POST

Autenticação vs Autorização

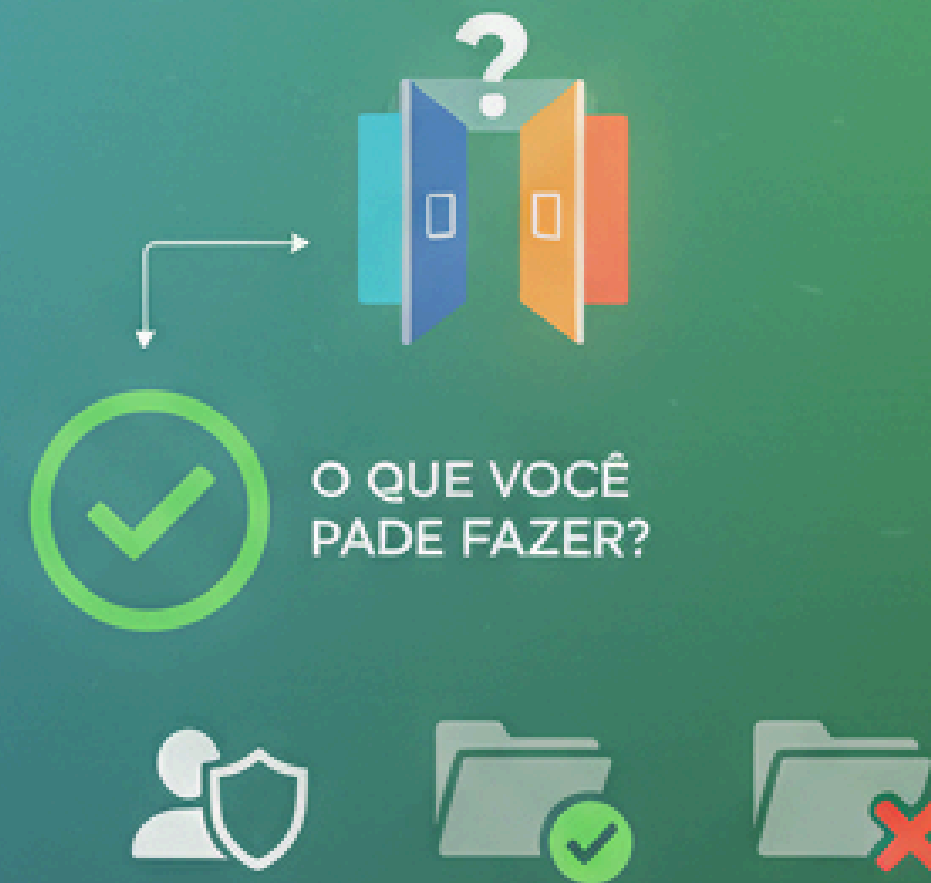
AUTENTICAÇÃO



VERIFICAÇÃO DE IDENTIDADE: USUÁRIO
+ SENHA, BIOMETRIA, ETC.

APÓS AUTENTICADO

AUTORIZAÇÃO



CONTROLE DE ACESSO: PERMISSÕES;
NÍVEIS DE USUÁRIO.

Autenticação → Quem é você ?

- Tem como objetivo comprovar a identidade de quem acessa o sistema
- Essa comprovação é feita através de credenciais, como email, senha, token de acesso
- Assim o sistema cria um contexto de usuário autenticado, através de sessão ou token

Autorização → O que você pode fazer ?

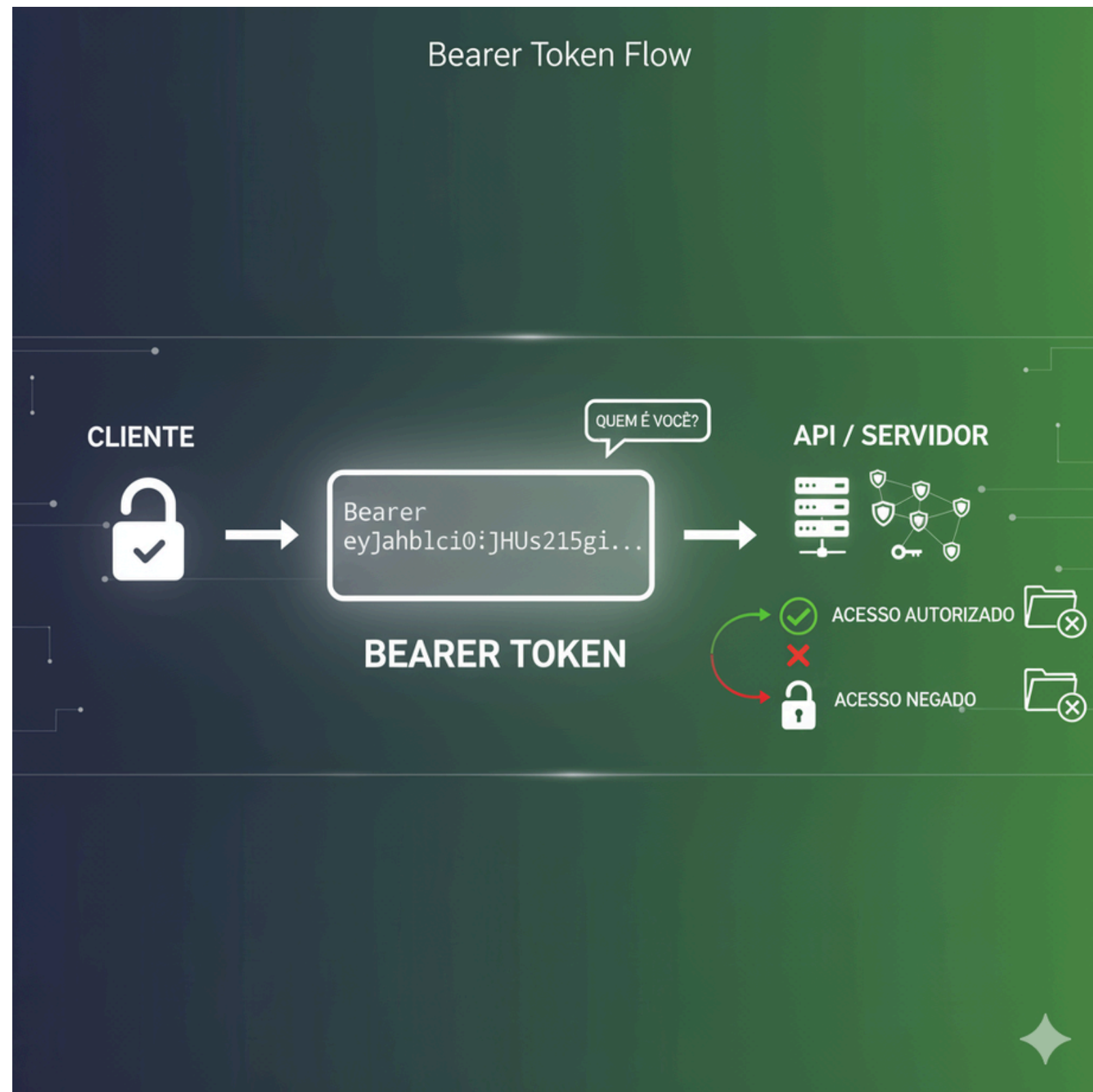
- Tem como objetivo decidir quais recursos ou ações o usuário pode ver/realizar após autenticado
- Isso é feito determinando ROLES para o usuário, ou seja os papéis do usuário no sistema (ADM, USER, etc.)

Spring Security



- É o framework responsável pela segurança de uma aplicação SpringBoot
- Faz a autenticação e autorização e também já trás proteções prontas, como header de segurança e configurações CORS
- Authentication: credenciais → um authentication autenticado é o usuário e suas permissões
- Authorization: decide o acesso de cada usuário
- Filter Chain: são os filtros que processam cada requisição
- UserDetails: identifica uma classe de um usuário que irá ser autenticado
- PasswordEncoder: responsável pelo hashing de senha

Token de autenticação



- É um comprovante digital de identidade e permissões, usado para acessar APIs
- Com ele não é necessário emitir uma senha a cada requisição
- Habilita autenticação e autorização em modo stateless
- O principal em aplicações SpringBoot é o JWT
- Bearer → é o esquema de autenticação HTTP, onde quem possui um token válido é tratado como autenticado

Stateless VS Stateful

Stateful → com estado no servidor

- O servidor guarda sessão após o login
- O cliente recebe um cookie de sessão
- O servidor consulta a sessão para identificar o usuário e suas permissões

Stateless → sem estado no servidor

- O servidor não guarda sessão após o login, cada requisição carrega o que precisa
- O cliente envia um token, geralmente um BEARER JWT, no header
- O servidor valida o token e extrai as claims

JWT Token → JSON WEB TOKEN

- É um comprovante digital em formato JSON
- É usado para autenticação e autorização em aplicações stateless

Estrutura:

- Header → metadados
- Payload → informações como identidade, permissões e expiração
- Signature → garante integridade e autenticidade do token

API:

- O usuário se autentica e recebe um access token
- O cliente envia o token em todas requisições protegidas no cabeçalho:
Authorization: Bearer <token>
- O Spring valida o token e autoriza o acesso com base nas claims

BCrypt



- É um algoritmo de hash projetado para armazenar senhas de forma segura
- Diferente dos hash padrões, é lento e ajustável, dificultando ataques brute-force
- O sistema não salva a senha puramente em texto, a apenas guarda o hash BCrypt dela
- Não é possível desfazer o hash para se obter a senha original
- O Spring fornece um PasswordEncoder baseado em BCrypt
- Na autenticação, ele faz `matches(rawPassword, storedHash)` — sem expor a senha real